BÁO CÁO THỰC HÀNH EMBEDDED LINUX – LAB 01

Xây dựng hệ thống Linux tối giản cho ARM Versatile Express và khởi động trên QEMU

Sinh viên	Hoang Trung Hai
MSSV	SS204779
Môn học	Embedded Linux
Lab	Lab 01
Nền tảng	ARM Versatile Express Cortex-A9 / QEMU
Linux Kernel	5.15
U-Boot	2026.04-LTS
BusyBox	1.35.0

1. Mục tiêu thực hành

Mục tiêu của Lab 01 là xây dựng một hệ thống Embedded Linux tối giản cho nền tảng ARM Versatile Express Cortex-A9. Quá trình thực hành bao gồm cấu hình và biên dịch Linux Kernel 5.15, xây dựng U-Boot 2026.04-LTS, biên dịch BusyBox 1.35.0 và tạo initramfs. Cuối cùng, toàn bộ hệ thống được khởi động bằng QEMU và kiểm tra kernel, root filesystem, các filesystem ảo và tiến trình hệ thống.

2. Môi trường và công cụ

Môi trường phát triển sử dụng Ubuntu 26.04 LTS 64-bit. Hệ thống sử dụng cross-compiler arm-linux-gnueabihf- để tạo mã máy ARM 32-bit. QEMU được sử dụng để mô phỏng máy vexpress-a9 với CPU Cortex-A9.

Các thành phần chính:

Thành phần	Phiên bản / cấu hình
Linux Kernel	5.15
U-Boot	2026.04-LTS
BusyBox	1.35.0
Architecture	ARMv7 / Cortex-A9
QEMU machine	vexpress-a9
Memory	512 MB

Console	ttyAMA0, 115200 baud

3. Cấu trúc dự án

```
embedded_lab1/
■■■configs/
■   ■■■kernel.config
■   ■■■busybox.config
■■■output/
■   ■■■zImage
■   ■■■vexpress-v2p-ca9.dtb
■   ■■■u-boot
■   ■■■initramfs.cpio.gz
■■■rootfs/
■   ■■■initramfs/
■■■kernel/
■   ■■■linux-5.15/
■■■uboot/
■   ■■■u-boot-2022.04/
■■■rootfs/
■   ■■■busybox-1.35.0/
■■■bao_cao/
    ■■■MSSV_Lab01_BaoCao.pdf
```

Các source tree lớn và build tree được giữ trong máy phát triển nhưng được loại khỏi Git repository bằng .gitignore. Repository tập trung vào các cấu hình, root filesystem, các output cần nộp và báo cáo.

4. Boot sequence

Chuỗi khởi động của hệ thống được mô tả theo thứ tự: Power ON → ROM bootloader → U-Boot → Linux Kernel → init → BusyBox shell.

Giai đoạn	Thành phần	Công việc chính	Dữ liệu / kết quả
1	Power ON / ROM	Khởi tạo phần cứng tối thiểu và chuyển quyền điều khiển cho bootloader.	CPU bắt đầu thực thi mã khởi động.
2	U-Boot	Khởi tạo môi trường boot và chuẩn bị kernel image và DTB.	Kernel image và DTB được nạp vào RAM.
3	Linux Kernel	Khởi tạo CPU, memory, driver, scheduler và các subsystem.	Kernel 5.15 bắt đầu chạy trên ARMv7.
4	Initramfs	Kernel giải nén root filesystem vào RAM.	Root filesystem chứa BusyBox và /init.
5	init	PID 1 thực hiện khởi tạo userspace.	Tiến trình init trở thành PID 1.
6	BusyBox shell	Cung cấp shell và các tiện ích Unix cơ bản.	Người dùng nhận được prompt / #.

5. Part A – Cấu hình và build Linux Kernel

Linux Kernel 5.15 được cấu hình cho kiến trúc ARM và nền tảng Versatile Express. Cấu hình cuối cùng có các thành phần ARMv7 cần thiết như ARM architecture, multi-platform ARMv7, CPU v7, EABI và module support. Kernel được biên dịch bằng cross-compiler arm-linux-gnueabihf-.

Các file output chính:

```
output/zImage
output/vexpress-v2p-ca9.dtb
```

Kiểm tra file:

```
$ file output/zImage
Linux kernel ARM boot executable zImage (kernel >=v4.15) (little-endian)

$ file output/vexpress-v2p-ca9.dtb
Device Tree Blob version 17, size=14081, boot CPU=0, ...
```

Kích thước output đã kiểm tra:

```
zImage                  4.9M
vexpress-v2p-ca9.dtb    14K
```

Kết quả file cho thấy zImage là Linux kernel executable dành cho ARM, trong khi DTB là Device Tree Blob được dùng để truyền thông tin phần cứng cho kernel.

6. Part B – U-Boot

U-Boot 2022.04 được cấu hình và build cho nền tảng ARM. U-Boot đóng vai trò bootloader, có nhiệm vụ khởi tạo môi trường boot, thiết lập boot arguments và chuẩn bị kernel/device tree trước khi chuyển quyền điều khiển cho Linux kernel.

```
output/u-boot
ELF 32-bit LSB executable, ARM, EABI5, static-pie linked,
with debug_info, not stripped
Kích th■ c: kho■ng 4.2M
```

Một kiểm tra quan trọng trong Part B là quan sát hành vi khi kernel không có root filesystem. Kernel có thể khởi động đến giai đoạn mount root filesystem nhưng không thể hoàn tất userspace nếu không tìm thấy init hoặc root device phù hợp. Đây là cơ sở để phân biệt nhiệm vụ của kernel với nhiệm vụ của root filesystem.

7. Part C – BusyBox và initramfs

BusyBox 1.35.0 được build cho ARM và cài đặt vào rootfs/initramfs. Root filesystem chứa /init cùng các thư mục userspace cần thiết như /bin, /sbin, /etc, /dev, /proc, /sys, /tmp và /usr. Initramfs sau đó được đóng gói thành file gzip để truyền cho QEMU.

```
output/initramfs.cpio.gz
Kích th■■c: 1017K

Root filesystem:
bin  dev  etc  init  lib  proc  root  sbin  sys  tmp  usr
```

Initramfs là filesystem tạm thời nằm trong RAM, được kernel sử dụng trong giai đoạn userspace ban đầu. Trong bài thực hành này, /init khởi tạo môi trường userspace và đưa người dùng vào BusyBox shell.

8. Khởi động hệ thống bằng QEMU

Lệnh QEMU sử dụng trong bài:

```
qemu-system-arm \
    -M vexpress-a9 \
    -cpu cortex-a9 \
    -m 512M \
    -nographic \
    -kernel output/zImage \
    -dtb output/vexpress-v2p-ca9.dtb \
    -initrd output/initramfs.cpio.gz \
    -append "console=ttyAMA0,115200 rdinit=/init"
```

Trong cấu hình trên, -M vexpress-a9 chọn máy mô phỏng Versatile Express, -cpu cortex-a9 chọn CPU Cortex-A9, -m 512M cấp 512 MB RAM, -kernel chỉ tới zImage, -dtb truyền Device Tree Blob, -initrd truyền initramfs và -append thiết lập console cùng chương trình init userspace.

9. Kết quả boot và kiểm tra

Kernel identification:

```
/ # uname -a
Linux embedded-lab 5.15.0 #4 SMP Mon Sep 14 10:24:45 +07 2026 armv7l GNU/Linux
```

Kết quả uname xác nhận kernel 5.15.0 đang chạy trên kiến trúc armv7l. Hệ thống nhận CPU ARMv7 và kernel được khởi động thành công.

Filesystem:

```
/ # mount
rootfs on / type rootfs (rw,size=244928k,nr_inodes=61232)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /proc type proc (rw,relatime)
none on /tmp type tmpfs (rw,relatime)
```

Kết quả cho thấy rootfs đã được mount và các filesystem ảo proc, sysfs và tmpfs hoạt động. Việc /proc và /sys có thể truy cập là dấu hiệu userspace đã được khởi tạo đúng ở mức cơ bản.

Process:

```
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:00 init
    2 root      0:00 [kthreadd]
   ...
   60 root      0:00 -/bin/sh
   63 root      0:00 ps
```

PID 1 là init, xác nhận tiến trình userspace đầu tiên đã chạy. BusyBox shell cũng hoạt động và lệnh ps có thể liệt kê tiến trình kernel/userspace.

Root filesystem:

```
/ # ls /
bin   dev   etc   init   lib   proc   root   sbin   sys   tmp   usr
```

Cấu trúc root filesystem phù hợp với một hệ thống Linux tối giản sử dụng BusyBox. Các thành phần quan trọng gồm /init, các thư mục binary, device nodes và filesystem ảo.

10. Phân tích và trả lời câu hỏi

Câu 1. Vai trò của Linux Kernel, U-Boot và BusyBox khác nhau như thế nào?

U-Boot là bootloader, hoạt động trước kernel và chuẩn bị môi trường để kernel được nạp và chạy. Linux Kernel quản lý CPU, memory, scheduler, driver, filesystem và cung cấp các cơ chế hệ điều hành. BusyBox hoạt động ở userspace và cung cấp shell cùng nhiều tiện ích Unix trong một binary nhỏ gọn. Do đó ba thành phần nằm ở các lớp khác nhau của chuỗi boot và hệ điều hành.

Câu 2. Vì sao cần Device Tree Blob?

Device Tree Blob chứa mô tả phần cứng mà kernel cần trong quá trình khởi động, chẳng hạn thông tin về CPU, memory, peripheral và các thiết bị liên quan. Với hệ thống ARM embedded, DTB giúp tách mô tả phần cứng khỏi mã kernel và cho phép cùng một kernel image được sử dụng với các cấu hình phần cứng khác nhau.

Câu 3. Initramfs có vai trò gì?

Initramfs cung cấp root filesystem ban đầu trong RAM. Kernel giải nén initramfs và tìm chương trình init để chuyển từ kernel space sang userspace. Trong Lab 01, initramfs chứa BusyBox và /init nên hệ thống có thể đi thẳng vào BusyBox shell mà không cần một filesystem root trên ổ đĩa vật lý.

Câu 4. Điều gì xảy ra nếu không có root filesystem hoặc không tìm thấy init?

Kernel vẫn có thể hoàn thành nhiều bước khởi tạo phần cứng và subsystem, nhưng không thể hoàn tất quá trình chuyển sang userspace nếu không mount được root filesystem hoặc không tìm thấy chương trình init. Trường hợp này thường dẫn đến kernel panic với thông báo liên quan đến việc không tìm thấy init hoặc không mount được root filesystem.

Câu 5. Vì sao dùng cross-compilation?

Máy phát triển là hệ thống x86-64 trong khi mục tiêu là ARMv7. Cross-compiler arm-linux-gnueabihf- cho phép biên dịch mã nguồn trên máy x86-64 thành executable chạy trên ARM. Cách này tránh việc phải thực hiện toàn bộ quá trình build trực tiếp trên một board ARM có tài nguyên hạn chế.

Câu 6. Ý nghĩa của kết quả uname, mount và ps?

uname xác nhận kernel version và kiến trúc CPU; mount xác nhận root filesystem và filesystem ảo đã được gắn; ps xác nhận userspace process đang chạy, đặc biệt PID 1 là init. Khi ba kiểm tra này đều đạt, có thể xác nhận không chỉ kernel đã boot mà userspace tối thiểu cũng đã hoạt động.

11. Khó khăn và cách giải quyết

Khó khăn 1 – cấu hình và cross-compilation cho ARM: Trong quá trình build, cấu hình kernel phải phù hợp với ARMv7 thay vì cấu hình host x86. Giải pháp là kiểm tra architecture/configuration, sử dụng đúng cross-compiler arm-linux-gnueabihf- và xác nhận

output bằng lệnh file.

Khó khăn 2 – phân biệt kernel image, DTB và root filesystem: Việc kernel build thành công chưa đồng nghĩa hệ thống có thể vào shell. Giải pháp là tạo initramfs chứa BusyBox và /init, truyền đồng thời zImage, DTB và initramfs cho QEMU, sau đó kiểm tra uname, mount, ps và ls /.

Khó khăn 3 – quản lý repository: Source tree của Linux, U-Boot và BusyBox rất lớn nên không phù hợp để đưa toàn bộ vào repository nộp bài. Giải pháp là sử dụng .gitignore để loại source/build tree và chỉ quản lý configs, rootfs, output cần thiết và báo cáo.

12. Kết luận

Lab 01 đã hoàn thành chuỗi xây dựng một hệ thống Embedded Linux tối giản trên nền ARM Versatile Express. Linux Kernel 5.15 được build thành công dưới dạng ARM zImage, Device Tree Blob được tạo cho vexpress, U-Boot 2022.04 được build cho ARM và BusyBox 1.35.0 được đóng gói thành initramfs. Hệ thống sau đó được khởi động thành công trên QEMU vexpress-a9.

Kết quả uname xác nhận kernel ARMv7 đang chạy; mount xác nhận rootfs, proc, sysfs và tmpfs; ps xác nhận init là PID 1 và BusyBox shell hoạt động. Các kết quả này chứng minh toàn bộ chuỗi từ kernel đến userspace đã được thiết lập và kiểm tra thành công.

13. Các file output nộp bài

File	Mô tả
output/zImage	Linux Kernel 5.15 ARM boot image
output/vexpress-v2p-ca9.dtb	Device Tree Blob cho Versatile Express
output/u-boot	U-Boot 2022.04 ARM executable
output/initramfs.cpio.gz	BusyBox initramfs
configs/kernel.config	Cấu hình Linux Kernel
configs/busybox.config	Cấu hình BusyBox
bao_cao/MSSV_Lab01_BaoCao.pdf	Báo cáo Lab 01

Lưu ý: Các trường Sinh viên và MSSV ở trang đầu đang để trống để điền đúng thông tin cá nhân trước khi nộp.